K.R. Mangalam University

School of Engineering & Technology
Operating System Lab (ENCA252)
Lab Assignment 2
Implementation of Banker's Algorithm for Deadlock Avoidance

Submitted by:
Name: Anurag Kumar Mishra

Roll Number: 2401201076

Course: BCA (AI & DS)

Submitted To:
Dr. Jyoti Yadav

1. Objective

The aim of this lab exercise is to implement Banker's Algorithm — a well-known deadlock avoidance technique used in operating systems — and use it to determine whether a given system state is safe or unsafe. If a safe state exists, the algorithm also identifies the safe execution sequence for all processes.

2. Tools & Technologies Used

Tool	Purpose
Python 3.x	Primary language used for implementation
Built-in modules: os, multiprocessing, time	Standard utilities used in the project
Linux / macOS Terminal	Platform used to run and verify the program
GitHub	Repository hosting and version tracking

3. Key Concepts

Term	Formula / Description
Allocation Matrix	Resources currently held by each process
Maximum Matrix	Maximum resources a process may ever demand
Need Matrix	Need = Maximum - Allocation
Available Vector	Resources that are currently free in the system
Work Vector	Simulation copy of Available; updated during the safety check
Safe State	A state where at least one safe sequence exists for all processes
Safe Sequence	Order in which all processes can complete without risking deadlock

4. Task 1 — System Input and Data Representation

System data is gathered interactively from the user: the number of processes, number of resource types, the Allocation Matrix, the Peak Demand (Maximum) Matrix, and the vector of currently available resources. All collected data is stored internally and displayed in a structured tabular format for verification.

Input Data Used

Number of Processes: 5 **Number of Resource Types:** 3

Allocation Matrix

Process	R0	R1	R2
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

Maximum Matrix

Process	R0	R1	R2
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2
P4	4	3	3

Available Resources: [3, 3, 2]

Screenshot — Input & Matrices Output

```
============================================================
   INPUT SUMMARY
============================================================

------------------------------------------------------------
  Allocation Matrix
------------------------------------------------------------
        R0   R1   R2
  P0   [  0    1    0 ]
  P1   [  2    0    0 ]
  P2   [  3    0    2 ]
  P3   [  2    1    1 ]
  P4   [  0    0    2 ]

------------------------------------------------------------
  Maximum Matrix
------------------------------------------------------------
        R0   R1   R2
  P0   [  7    5    3 ]
  P1   [  3    2    2 ]
  P2   [  9    0    2 ]
  P3   [  2    2    2 ]
  P4   [  4    3    3 ]

  Available Resources: [3, 3, 2]
```

Terminal showing entered matrices and available resources

5. Task 2 — Need Matrix Calculation

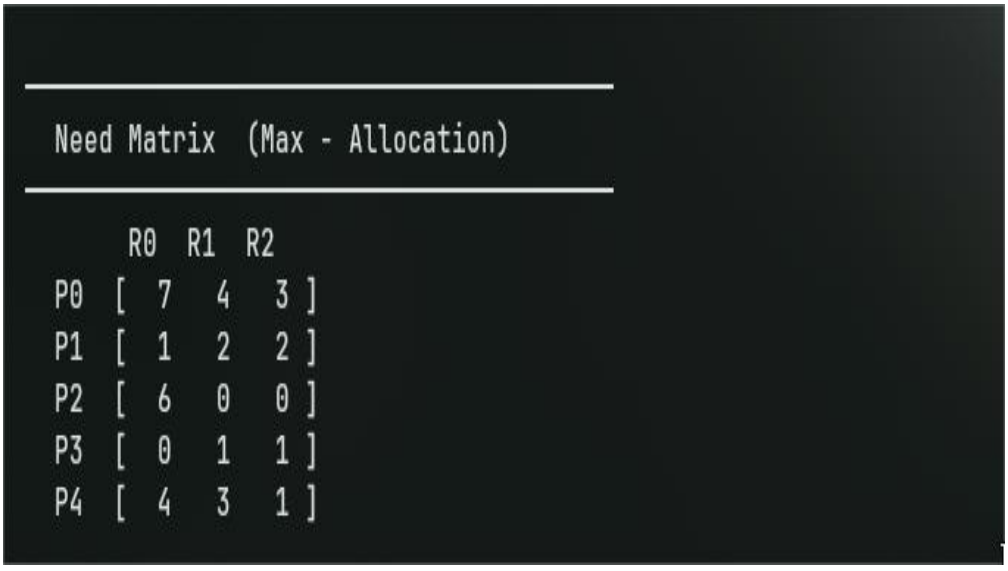
The **Need Matrix** captures how many additional resources each process may still request before it finishes executing. It is derived from the Allocation and Maximum matrices using the formula below:

```
Need[j]  =  Maximum[i][j]  -  CurrentAllocationn[i][j]
```

Need Matrix

Process	R0	R1	R2
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1

Screenshot — Need Matrix Output



Terminal displaying the computed Remaining Need matrix

6. Task 3 — Banker's Safety Algorithm Implementation

The Safety Algorithm simulates resource allocation to determine if the system is in a **safe state**. It scans processes repeatedly and selects those whose remaining need can be satisfied by the current buffer, then simulates their completion and resource release.

1. **Initialise Buffer** = copy of Available resources
2. **Set Completed[i]** = False for every process
3. Locate a process i where: `Completed[i] == False` AND `Need[i] <= Buffer`
4. Simulate completion: `Buffer = Buffer + Allocation[i]`, set `Completed[i] = True`
5. Repeat steps 3-4 until all processes complete or no eligible process remains
6. If all `Completed[i] == True` --> **Safe State**, else --> **Unsafe State**

Step-by-Step Trace

Step	Process	Need	Buffer (before)	Buffer (after)
1	P1	[1, 2, 2]	[3, 3, 2]	[5, 3, 2]
2	P3	[0, 1, 1]	[5, 3, 2]	[7, 4, 3]
3	P4	[4, 3, 1]	[7, 4, 3]	[7, 4, 5]
4	P0	[7, 4, 3]	[7, 4, 5]	[7, 5, 5]
5	P2	[6, 0, 0]	[7, 5, 5]	[10, 5, 7]

Screenshot — Safety Algorithm Trace Output

```
============================================================
   SAFETY ALGORITHM — STEP-BY-STEP TRACE
============================================================
  Initial Work (Available) : [3, 3, 2]

  Step 1: P1 can proceed
    Need     : [1, 2, 2]
    Work     : [3, 3, 2]  →  [5, 3, 2]

  Step 2: P3 can proceed
    Need     : [0, 1, 1]
    Work     : [5, 3, 2]  →  [7, 4, 3]

  Step 3: P0 can proceed
    Need     : [7, 4, 3]
    Work     : [7, 4, 3]  →  [7, 5, 3]

  Step 4: P2 can proceed
    Need     : [6, 0, 0]
    Work     : [7, 5, 3]  →  [10, 5, 5]

  Step 5: P4 can proceed
    Need     : [4, 3, 1]
    Work     : [10, 5, 5]  →  [10, 5, 7]
```

Terminal showing step-by-step safety algorithm execution

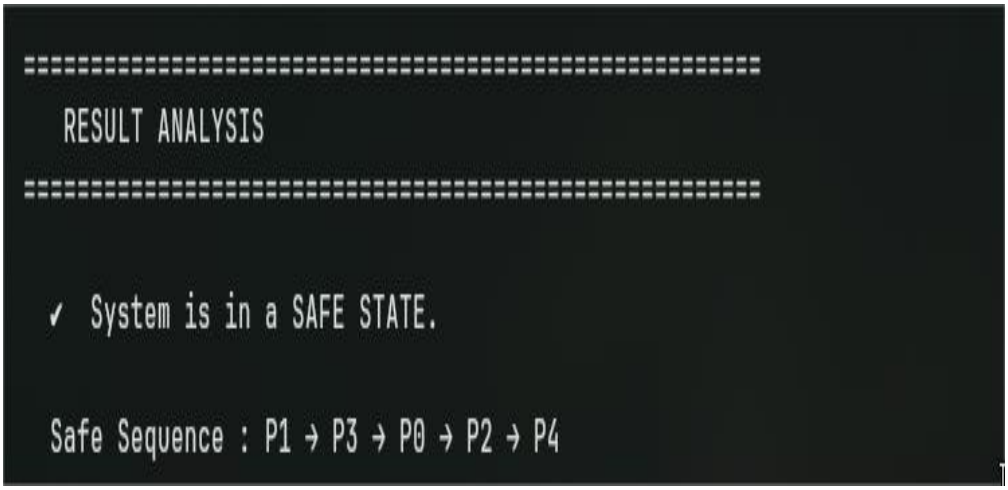
7. Task 4 — Safe Sequence Determination

Because every process was successfully scheduled during the safety check, the system is confirmed to be in a **safe state**. The following safe execution sequence was identified:

```
P1  -->  P3  -->  P4  -->  P0  -->  P2
```

Executing processes in this order guarantees that each one will always find sufficient resources available — after previously completed processes have returned their allocations to the pool — and therefore no process ever needs to wait indefinitely for resources.

Screenshot — Safe Sequence Output



Terminal displaying the final safe sequence verdict

8. Task 5 — Result Analysis

System State Verdict

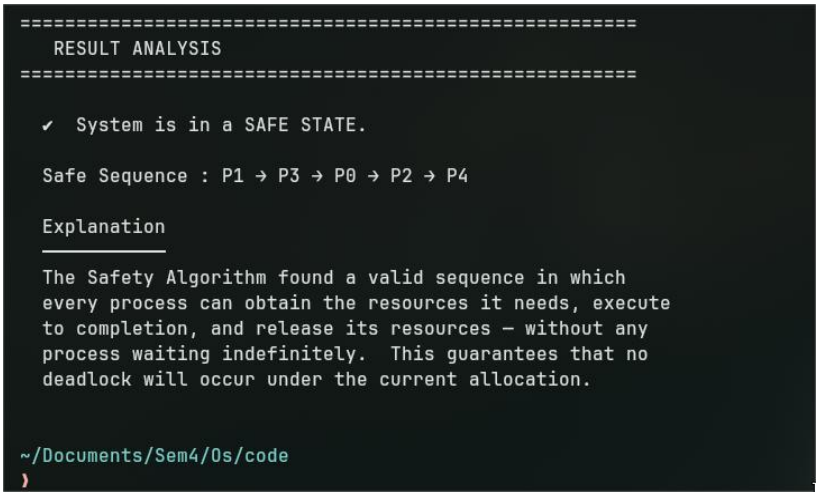
Parameter	Value
System State	SAFE
Safe Sequence Found	P1 --> P3 --> P4 --> P0 --> P2
Total Processes	5
Total Resource Types	3

Analysis

Comparison with Theoretical Expectations

- The safety check successfully identified a complete execution order for all 5 processes, confirming the system is deadlock-free.
- At every step, the selected process had its **Remaining Need <= Buffer**, meaning it could acquire all required resources and proceed to completion.
- Once each process finished, it returned its allocated resources back into the buffer, enlarging the pool available for subsequent processes.
- Since all `Completed[i]` values became True, the system is confirmed to be in a **safe state** — no deadlock can occur under the current allocation.
- The outcome aligns with the expected result from the standard textbook example of Banker's Algorithm.
- A safe state theoretically guarantees the OS can always find at least one valid process ordering that avoids deadlock — which is precisely what was observed here.
- Had the available resources been reduced (e.g., Available = [1, 0, 0]), no safe sequence would have existed and the system would have entered an unsafe state, risking deadlock.

Screenshot — Final Result Output



```
=========================================================
   RESULT ANALYSIS
=========================================================

  ✔  System is in a SAFE STATE.

  Safe Sequence : P1 → P3 → P0 → P2 → P4

  Explanation
  ──────────
  The Safety Algorithm found a valid sequence in which
  every process can obtain the resources it needs, execute
  to completion, and release its resources — without any
  process waiting indefinitely.  This guarantees that no
  deadlock will occur under the current allocation.


~/Documents/Sem4/Os/code
❯
```

Terminal showing final safe state verdict and execution sequence

9. GitHub Repository

Link: https://github.com/anuragkumarmishra2005/Operating-System-Lab-Assignment-2

10. Source Code

The complete source code is included below. It is also available in the linked GitHub repository under the filename *anurag_mishra_os_assignment2.py*.

```
"""
================================================================
  Deadlock Avoidance using Banker's Algorithm
  Course  : Operating System Lab (ENCA252)
  Program : BCA (AI & DS)
  Name    : Anurag Kumar Mishra
  Roll No : 2401201076
================================================================
"""

import os

# ================================================================
# SECTION 1 — System Input & Data Collection
# ================================================================

def read_system_data():
    num_proc = int(input("Total number of processes        : "))
    num_res  = int(input("Total number of resource types   : "))

    print(f"\n[Allocation Table] — {num_proc} processes x {num_res} resource types")
    alloc_table = []
    for i in range(num_proc):
        row = list(map(int, input(f"  Process {i} : ").split()))
        alloc_table.append(row)

    print(f"\n[Peak Demand Table] — {num_proc} processes x {num_res} resource types")
    peak_table = []
    for i in range(num_proc):
        row = list(map(int, input(f"  Process {i} : ").split()))
        peak_table.append(row)

    print(f"\n[Free Resources] — enter {num_res} space-separated values:")
    free_res = list(map(int, input("  Free : ").split()))

    return {
        "num_proc"   : num_proc,
        "num_res"    : num_res,
        "alloc_table": alloc_table,
        "peak_table" : peak_table,
        "free_res"   : free_res,
    }

def show_matrix(label, matrix, num_proc, num_res):
    print(f"\n{'~' * 42}")
    print(f"  {label}")
    print(f"{'~' * 42}")
    col_header = "        " + "   ".join([f"R{j}" for j in range(num_res)])
    print(col_header)
    for i in range(num_proc):
        vals = "   ".join(str(v).rjust(2) for v in matrix[i])
        print(f"  Proc{i}  [ {vals} ]")
```

```
def show_vector(label, vec):
    print(f"\n  {label} : {vec}")

# ================================================================
# SECTION 2 — Remaining Need Matrix Calculation
# ================================================================

def compute_remaining(peak_table, alloc_table, num_proc, num_res):
    remaining = []
    for i in range(num_proc):
        row = [peak_table[i][j] - alloc_table[i][j] for j in range(num_res)]
        remaining.append(row)
    return remaining

# ================================================================
# SECTION 3 — Safety Check Algorithm
# SECTION 4 — Safe Execution Sequence
# ================================================================

def check_safety(num_proc, num_res, alloc_table, remaining, free_res):
    buffer = free_res[:]
    done   = [False] * num_proc
    order  = []
    step   = 0

    print(f"\n  Starting Buffer (Free Resources) : {buffer}\n")

    while len(order) < num_proc:
        progress = False
        for i in range(num_proc):
            if done[i]:
                continue
            can_run = all(remaining[i][j] <= buffer[j] for j in range(num_res))
            if can_run:
                prev_buffer = buffer[:]
                buffer  = [buffer[j] + alloc_table[i][j] for j in range(num_res)]
                done[i] = True
                order.append(i)
                progress = True
                step += 1
                print(f"  Step {step}: Process {i} scheduled")
                print(f"    Remaining Need : {remaining[i]}")
                print(f"    Buffer Before  : {prev_buffer}")
                print(f"    Buffer After   : {buffer}\n")
                break
        if not progress:
            break

    return all(done), order

# ================================================================
# SECTION 5 — Output, Analysis & Verdict
# ================================================================

def show_verdict(is_safe, order, num_proc):
    if is_safe:
        seq = " --> ".join([f"P{p}" for p in order])
        print(f"\n  [SAFE]  System is currently in a SAFE STATE.")
        print(f"\n  Execution Order : {seq}")
    else:
        scheduled = set(order)
        blocked   = [f"P{i}" for i in range(num_proc) if i not in scheduled]
        partial   = " --> ".join([f"P{p}" for p in order]) if order else "None"
        print(f"\n  [UNSAFE]  System is in an UNSAFE STATE.")
        print(f"\n  Partial Sequence         : {partial}")
        print(f"  Processes that are stuck : {blocked}")

def main():
    os.system("clear" if os.name == "posix" else "cls")
    data        = read_system_data()
    num_proc    = data["num_proc"]
    num_res     = data["num_res"]
    alloc_table = data["alloc_table"]
```

```
    peak_table  = data["peak_table"]
    free_res    = data["free_res"]

    show_matrix("Allocation Table",  alloc_table, num_proc, num_res)
    show_matrix("Peak Demand Table", peak_table,  num_proc, num_res)
    show_vector("Free Resources",    free_res)

    remaining = compute_remaining(peak_table, alloc_table, num_proc, num_res)
    show_matrix("Remaining Need  (Peak - Allocated)", remaining, num_proc, num_res)

    is_safe, order = check_safety(num_proc, num_res, alloc_table, remaining, free_res)
    show_verdict(is_safe, order, num_proc)

if   name   == "  main  ":
    main()
```